

Modeling and implementing a real-time scheduler for dual-armed cluster tools

Yong-Ho Shin, Tae-Eog Lee^{*}, Ja-Hee Kim, Hwan-Yong Lee

Department of Industrial Engineering, KAIST, 373-1 Gusung-dong, Yuseong-Gu, Taejeon 305-701, South Korea

Abstract

As application of cluster tools is being extended to more wafer fabrication processes including chemical vapor deposition processes as well as etching processes, cluster tool configurations and wafer flow patterns tend to be more complicated. Frequent recipe changes require prompt adaptation of the scheduling logic to cope with changed configurations and wafer flow patterns. The scheduler should be responsive to occurrences of various events within the tool and should be able to coordinate and control complicated activities and tasks of the process modules and the wafer-handling robot. Moreover, the scheduler should meet a critical timing constraint that a wafer should leave the process module within a specified time limit after processing. Therefore, it is essential to have a systematic, flexible and agile method of developing a robust real-time scheduler. To do this, we propose a model-based approach based on finite state machines (FSMs) to develop a real-time scheduler for dual-armed cluster tools. We propose the scheduling decisions for normal operation. We then develop a scheduler model based on FSMs that specifies the scheduling decisions for normal operation and encodes real-time control decisions to cope with disturbances. We then verify the scheduler model and transform the model into an executable control code. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Cluster tool; Scheduling; Model; Finite state machine

1. Introduction

Recently, wafer fabrication equipments are adopting new technologies such as 300 mm wafer processing, mini-environment, standard mechanical interface (SMIF), and single wafer processing. Cluster tools are expected to lead such technology transitions. As conventional batch processes in etching, chemical vapor deposition (CVD), and rapid thermal processing are transformed into single wafer processing, cluster tools each of which integrates multiple single wafer processing modules (PMs) together into a cluster using a wafer handling robot have been developed.

A cluster tool consists of several single wafer PMs, a wafer handling robot, and two loadlocks for cassette loading and unloading, which are integrated into a closed, high-class clean room environment.

A cluster tool is operated by a control software called a *cluster tool controller* (CTC). The CTC manages the job data, communicates with the process modules and the wafer handling robot, and coordinates their activities. The subsystem of the CTC that is responsible for coordinating the activities or determining the sequence and timings of the robot tasks and the process starts of the PMs is called a *scheduler*.

Scheduling or coordinating the activities of a cluster tool is often challenging. There are several scheduling requirements. First, the wafer handling robot and the process modules are tightly coupled in a limited space.

^{*} Corresponding author.

E-mail address: telee@mail.kaist.ac.kr (T.-E. Lee).

There is no intermediate space except the robot's hands for a wafer to wait for next processing between the PMs. Second, different wafer lots may have different wafer processing recipes, that is, different process steps and conditions such as the temperatures, pressures, and the mixture of the gases in the PMs. When the recipe changes, the process step that each PM performs may change and the PMs are logically reconfigured, and the wafer flow through the PMs changes accordingly. Third, there are several exceptional events, such as a parallel module breakdown and preventive cleaning of a process module, for which special handling procedures are required. Fourth, processing times or robot task times are subject to random fluctuation, and hence exact timing control using a predetermined schedule is difficult. Therefore, the scheduler should be responsive to various events that cause the system state change. Finally, since recent CVD cluster tools tend to use rapid thermal processing technologies, they have strict timing constraints that require a processed wafer at a PM which should leave the PM within a specified time limit. Otherwise, the wafer is subject to quality problems due to residual heat and gases in the PM's chamber. Such timing constraint is called the *residency time constraint*. There have been attempts to meet some of the scheduling requirements, including conventional scheduling methods based on pre-computed offline schedules like Gantt charts, or dispatching rules [1–7].

In this paper, we propose a model-based approach for designing and implementing a real-time scheduler for dual-armed cluster tools, which can meet the scheduling requirements. We propose the scheduling decisions for normal operation. We then develop a scheduler model based on FSMs that specifies the scheduling decisions for normal operation and encodes real-time control decisions to cope with disruptions such as unexpected events like a parallel PM breakdown or cleaning a PM for preventive maintenance, and time variation. Exceptions handling procedures as well as the normal operation procedures are appropriately inscribed into the states and transitions. The scheduler model is then verified and transformed into an executable control code.

The paper is organized as follows. Section 2 explains cluster tool configuration and operation. Section 3 proposes the scheduling strategies and

decisions for normal operation. Section 4 models the scheduler logic for normal operation and for real-time control to cope with disruptions. Section 5 explains how to implement the modeled scheduler. Section 6 discusses the test and debugging method.

2. Tool configurations and operation

CVD cluster tools are usually configured with six or eight modules in radial layouts (see Fig. 1(a) and (b)). The modules are process modules (PM), an aligner module (AL), a cooler module (CL), and a wafer handling robot. Two loadlocks for wafer loading and unloading are located behind AL and CL. The robot has dual opposite pan-type arms, and performs wafer transportation, and wafer loading and unloading. The angle between the two arms is always kept as 180° . PMs perform CVD processes and can be configured differently for various recipes. A loadlock is the place where a cassette with wafers is loaded and unloaded. A cassette has 25 wafers, which all have an identical recipe in this study. The robot unloads a wafer from the cassette in the loadlock and puts the wafer on AL and waits there until the position of wafer is correctly aligned by a vision control system so that a robot hand can grip the right position. After aligning in a few second, the robot picks the wafer back, and moves and loads into the PM for the first process step. Therefore, AL need not be considered as a separate process step and the aligning time can be included in the robot task time for unloading a wafer from the loadlock. Every wafer should be cooled at CL

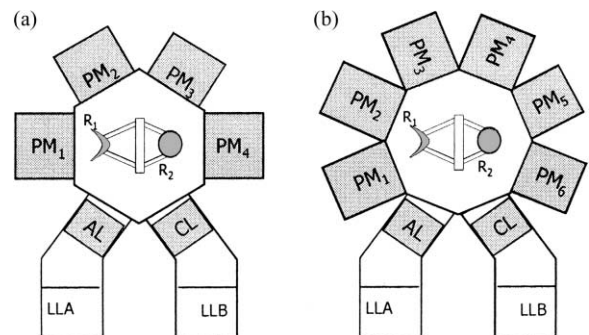


Fig. 1. Cluster tool configurations: (a) six-sided cluster tool; (b) eight-sided cluster tool.

before returning to the loadlock after finishing all process steps not to damage the other wafers in the loadlock due to the residual heat. A wafer unloaded from a PM that does not complete all process steps yet should not be cooled down in CL or the loadlock in order to keep the moderate temperature for the next process step and hence cannot be returned to the loadlock to wait for the next process step. After processing all wafers in the cassette in a loadlock, the robot starts processing for the wafers in the cassette at the other loadlock. During this time, the previous cassette with completed wafers is unloaded from the loadlock.

When a new wafer lot requires a different wafer processing recipe, the PMs are logically reconfigured to accommodate the new sequence of process steps and the wafer flow. The frequency of recipe changes and the configuration changes depends on the order characteristics, production planning, and the layout of the fabrication lines. Recent fabrication lines, not only for application specific integrated circuits (ASICs), but also for high-end memory chips, tend to have more frequent recipe changes. Generally, a cluster tool is responsible for five or more different recipes. Some PMs can be configured to perform an identical process in order to balance the workload among the processes. Such PMs are called *parallel*. The process engineer determines the process step that a PM will perform. For any cluster tool configuration, there is only one CL chamber because the cooling time is relatively short. Once the configuration is decided, the wafer flow pattern is determined based on the recipe and the load balance between the PMs. A wafer flow pattern is defined as follows. Every wafer, unloaded from the

loadlock and aligned at the AL, visits the PMs in the sequence of process steps specified in the recipe and finally the CL, and returns to the loadlock. When there are parallel PMs, a wafer visits one of them. The parallel PMs are visited in a cyclic order by the wafers. When wafers visit k parallel PMs and m serial PMs including CL (but excluding AL), the wafer flow pattern is denoted by $kPmS$. For instance, Fig. 2(a) illustrates a 6-sided cluster tool with 4S flow pattern where PM₁, PM₂, and PM₃ perform different processes and PM₄ is not used. Fig. 2(a) shows 2P4S flow pattern, where a wafer, unloaded from loadlock A (LLA) and aligned, visits either PM₁ or PM₂, and then PM₃, PM₄, and CL. The solid and dotted arrows indicate the wafer flow pattern from loadlock A and B, respectively. $mSkP$ represents the flow pattern such that the parallel PMs follow the serial PMs. Other flow patterns include $mSkPnS$, $jPmSkP$, and $mSjPkP$.

Even if one of the parallel PMs is aborted or unavailable due to occurrences of an alarm or a breakdown, the tool should continue processing using the other PMs. The wafer at the aborted PM gives up the next process and is immediately unloaded to CL, and returned to the loadlock immediately. Then, the aborted PM remains idle until the operator performs recovery actions. However, when a serial PM is aborted, the wafers in process at the aborted PM and its upstream PMs are similarly returned to the loadlock and the wafers at the downstream PMs complete their remaining processes. Then, the whole cluster tool keeps idle until the operator restores the aborted PM and resets the tool.

CVD cluster tools require the residency time constraint. Therefore, the residency time constraint

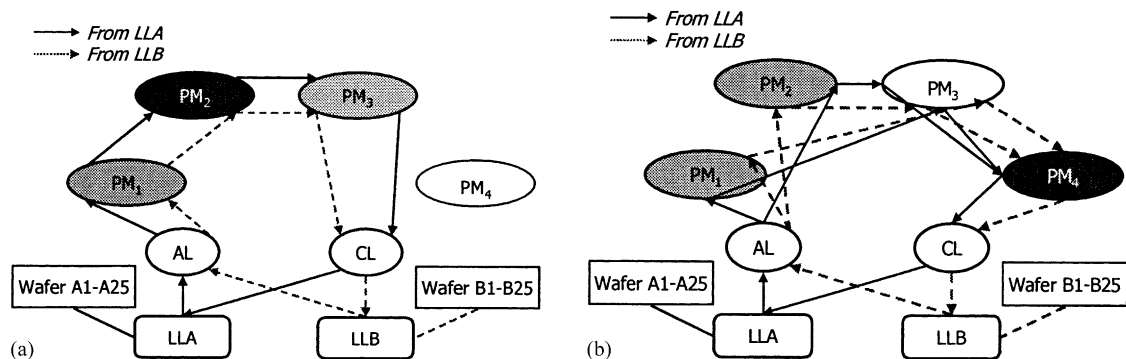


Fig. 2. Examples of wafer flow patterns: (a) 4S flow pattern; (b) 2P4S flow pattern.

together with the diverse wafer flow patterns and exception-handling procedures make the scheduling and control problem difficult. Furthermore, when a tool is used for research and development, more complicated operational modes are used.

3. Scheduling decisions for normal operation

In this section, the scheduling strategies and the scheduling decisions for normal operation will be explained, where there are no disruptions such as PMs or robot breakdowns, preventive maintenance of a PM, or process time variation. First, since the wafers in a cassette have an identical recipe and an identical visit sequence, there is no job sequencing issue at each PM. Second, the robot task sequence should be determined. Robot tasks include unloading a wafer from a PM or a loadlock, moving a wafer from one place to another place, and loading a wafer into a PM, CL, or a loadlock. The activities of a PM, such as starting processing and completing processing, follow the robot tasks.

There may be several scheduling strategies for robot tasks. A dispatching rule or priority-based rule selects one among the feasible robot tasks each time a robot task should be determined [3–7]. Some commercial cluster tool schedulers like ControlPro also use similar dispatching rules [4]. Many scheduling techniques for robot cells also would be extended for cluster tools with a dual-arm [8]. For dual-armed cluster tools, there is a useful scheduling strategy, called swap. We explain a scheduling decision based on the swap strategy for normal operation of a dual-armed cluster tool with the residency time constraint, such as a CVD cluster tool. A *swap operation* unloads a wafer processed at a module using the empty robot hand and loads the wafer on another robot hand into the module (see Fig. 3). Such swap operation is known to be efficient for dual-armed cluster tools (see [5]) and is adopted by many cluster tool vendors because it simplifies the scheduling problem and leads to predictable tool behavior. Once the tool uses the swap operation strategy, the robot task sequence is determined because the swap operations are repeated for the next PMs in the wafer visit sequence.

Therefore, the remaining decision is to determine the timings of the robot tasks. We use the *earliest*

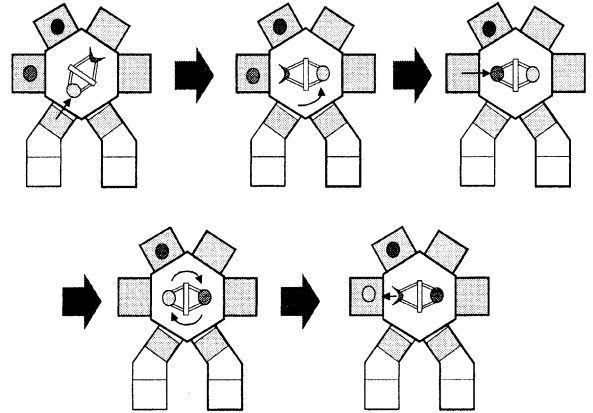


Fig. 3. Swap operation.

starting strategy except for the operations of unloading wafers from the loadlock. A wafer processed at a PM is unloaded as soon as possible if a robot arm is available. A wafer unloaded from a PM is loaded into the next PM as soon as possible if the next PM is available. The earliest starting method is easy to implement because we need not deliberately keep watching the task timings and computing the timings a priori [9]. Such strategy is appropriate especially when the robot tasks times and process times are subject to random variation. The time interval for which the robot unloads the next wafer from the loadlock is called the wafer delivery interval (WDI). The WDI is determined so that the maximum throughput is attained. WDI is determined as the cycle time as the bottleneck process. Fig. 4 illustrates the robot work cycle and the PM work cycle for 4S flow pattern. The left diagram shows a simplified graph model for the tool activities, where the detailed tasks or activities of the work cycles, listed at the right part, are appropriately aggregated on the arcs. Each loop represents a PM work cycle. The circuit connecting the loops is the robot work cycle.

The cycle time is determined as follows. The parallel PMs for a process step are identical. We let n be the number of process steps that each wafer should take. We let $M_1, M_i (i = 2, \dots, n-1)$ and M_{n+1} denote the robot move times from the loadlock to the PM for process step 1, from the PM for process step $(i-1)$ to the PM for process step $(i = 2, \dots, n-1)$ from the PM for process step n to CL, and from CL to the loadlock, respectively. $SW_i, i = 1, \dots, n$, indicate the

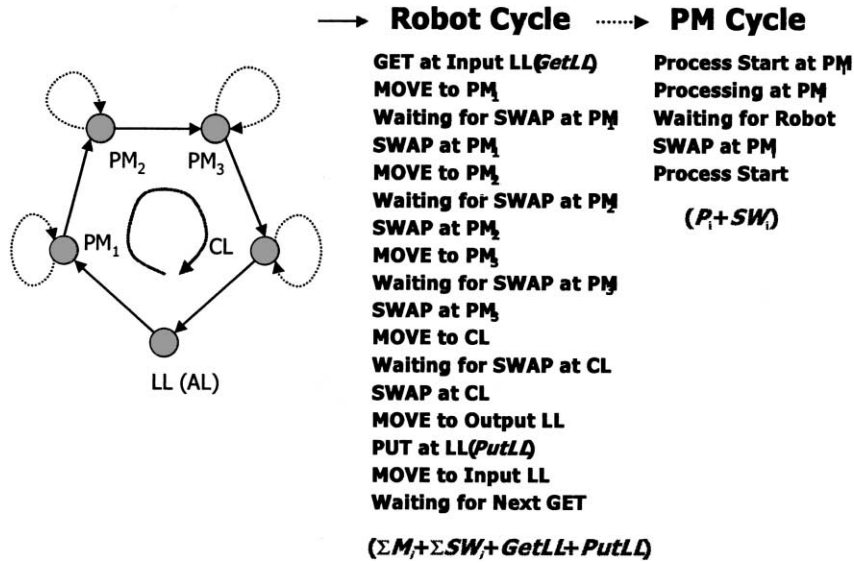


Fig. 4. The robot and PM work cycles for 4S wafer flow pattern.

swap operation times at the PMs for process step ($i = 2, \dots, n-1$), respectively. SW_{n+1} is the swap operation time at CL. For $i = 1, \dots, n$ P_i is the process time for process step i . P_{n+1} is the cooling time at CL. GetLL and PutLL represent the time for unloading a wafer from the loadlock including the aligning time and the time for returning a wafer from CL to the loadlock, respectively. As seen in Fig. 4, the directed graph model has a connecting circuit that corresponds to the robot work cycle, and $(n+1)$ loops (or circuits) for each PM and CL. Therefore, the cycle time is determined as the longest circuit. This also can be easily verified from the well-known result on the cycle time of an event graph, a special class of Petri nets, since the directed graph model can be converted into an event graph, a special class of Petri nets for which many useful mathematical properties are available [10]. Further, it can be seen that each circuit of the event graph has only one token. Therefore, the cycle time is determined by the longest circuit length. Consequently, we have the tool cycle time as

$$\max \left\{ \sum_{i=1}^{n+1} M_i \sum_{i=1}^{n+1} SW_i + GetLL + PutLL(P_1 + SW_1), \dots, (P_{n+1} + SW_{n+1}) \right\} \quad (1)$$

where the first term in the list corresponds to the robot work cycle, and the followings are for the work cycles for each process step and CL.

We note that the time deviations for GetLL and PutLL due to different wafer locations are negligible since the robot arm's rotation is fast. The waiting times in the robot work cycle occur when a PM is not ready for receiving a wafer from the robot or the process is not completed. Fig. 5 illustrates a Gantt chart schedule for a 4S flow pattern case. R_1 and R_2 represent the two robot hands. The boxes with the same shading indicate the activities and robot tasks for the same wafer. By following the boxes with the same shading from left to the right, we identify the sequence of activities and robot tasks that a wafer take. When the robot rotates the hands, both robot hands engage in the operation simultaneously. In Fig. 5, the cycle time by Eq. (1) is computed as

$$\max \{ (1 + 1 + 1 + 1) + (3 + 3 + 3 + 3) + 2 + 1, 20 + 3, 19 + 3, 19 + 3 \} = 23.$$

It is necessary to check whether the proposed scheduling method satisfies the residency time constraint for a given configuration, flow pattern, and the time data for the processes and the robot tasks. By some mathematical analysis, Lee [11,12] proposes

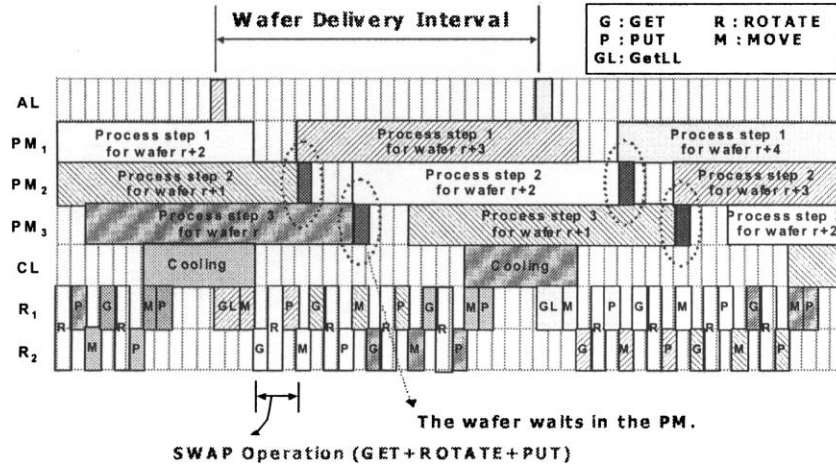


Fig. 5. A Gantt chart example for 4S wafer flow pattern.

simple schedulability checking inequalities for CVD cluster tools with most practical flow patterns when all swap times are identical:

$$\max \left\{ \frac{P_i + SW}{MP_i} \right\} \geq (n+1)M_i + n \times SW + \text{GetLL},$$

$$\text{and } \max \left\{ \frac{P_i + SW}{MP_i} \right\} - \left(\frac{P_i + SW}{MP_i} \right) \leq R_{i_{\max}}$$

for each i (2)

where MP_i is the number of parallel PMs for process step i , and $R_{i_{\max}}$ is the upper limit maximum on the residency time for process step i . The inequalities can be extended for the case where all swap times are not identical. The process engineers can accommodate the process times within some technical limits so as to satisfy the schedulability inequalities.

4. Modeling a real-time scheduler logic

The proposed scheduler model is based on FSMs and can specify not only the scheduling decisions for normal operation and but also real-time control decisions to cope with disturbances. Although we briefed the scheduling decisions for the normal situation in Section 3, there are real-time control requirements that the scheduler should accommodate the sequences and timings of the PM activities and robot tasks to cope with disruptions such as unexpected events and time

variation. First, the tasks or operations of the robot and PMs should be properly coordinated and controlled. Since the robot arms and PMs are tightly combined and interact closely, the activities or tasks are mutually related. Precedence, synchronization, and mutual exclusion are involved. Therefore, such coordination and control should have real-time control capabilities to cope with disruptions like unexpected events and time variations, not merely to meet the scheduling requirements for normal operation. Second, the process times and robot task times are subject to random variation due to communication delays or process control disturbances. Usual variance is 2–3%. Therefore, it is impractical to keep track of the pre-computed timings in a Gantt schedule. Frequent, time-consuming rescheduling or re-computing the timings may be unavoidable. Instead, it is reasonable to initiate a task or an activity when the message of an appropriate event occurrence is received. The scheduling decision or control should be responsive to the events from the robot and PMs. Third, there can be abnormal or exceptional events. For instance, one of parallel PMs may break down or a PM may fail to let the processed wafer leave within the residency time limit. For an occurrence of each predictable abnormal event, an appropriate recovery procedure is prepared. A PM should undergo a preventive maintenance called *auto-cleaning* when a specific technical condition is satisfied. Therefore, the scheduler should be able to easily accommodate such exception handling

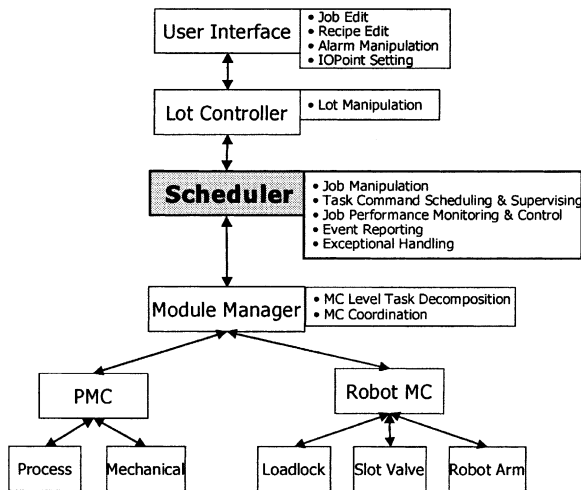


Fig. 6. Architecture of a cluster tool controller.

procedures. Fourth, since the configuration of the PMs and the wafer flow patterns change frequently depending on the recipes, the scheduler should be able to easily adapt the scheduling rule. It is not easy to memorize the Gantt chart schedules for all possibilities. Fifth, we wish that the tool behavior be more predictable. We prefer a scheduler development method that can implement the verified scheduling knowledge that exploits the structural properties of the cluster tool scheduling problem. Finally, the scheduler should be embedded and integrated into CTC. Fig. 6 shows the CTC architecture and the scheduler's role, which most commercial CTC software adopts.

The lot controller requests the scheduler to do job level tasks such as *JobStart*, *JobStop*, *JobAbort*, *JobPause*, and *JobResume*, which are encoded in function blocks. When a new cassette is loaded, the lot controller sends the scheduler *JobStart* command to retrieve the job information from the database of the CTC and to initialize the FSM. *JobAbort*, *JobPause*, and *JobResume* commands perform similar functions for aborting the job processing for a lot during processing and evacuating the tool, stopping the job processing temporarily, and resuming the paused job, respectively. Such job level management functions are often initiated by the operator. The scheduler executes the commands by manipulating the PMs and the robot as specified in the scheduling logic. The scheduler, embedded between the lot controller and the module manager of the CTC, performs

job manipulation, scheduling and supervising task commands, job performance monitoring and control, event reporting, and exceptional handling. Therefore, the scheduler should be implemented so that it can be easily integrated with CTC. Process module controller (PMC) and Robot MC are module controllers each of which monitors and controls a PM or the robot.

To meet the real-time control requirements on the cluster tool scheduler, we propose an approach based on formal models of the desirable scheduling behavior. Formal models such as FSMs or Petri nets have been used for modeling, scheduling, or controlling manufacturing systems [13–16]. Automata-based representations have been used for plug-and-play interoperability specification in equipment level standards for semiconductor manufacturing (for instance, see [17]). Following such model-based approaches, we develop a real-time scheduler logic model based on FSMs that not only can make scheduling decisions for normal operation but also can flexibly and dynamically determine appropriate real-time control decisions depending on the tool state, which handle disruptions such as a parallel PM breakdown, cleaning a PM for preventive maintenance, an alarm occurrence at a PM due to exceeding the residency time, or process time variation. Once the scheduler logic is modeled, we can verify the scheduling logic and make the FSM model into an executable control code to implement the scheduler. Exceptions handling procedures as well as the normal operation procedure are appropriately inscribed into the states and transitions.

First, the work cycle of each PM is modeled using a FSM. A FSM consists of states, transitions, and events. An event triggers a transition from a state to another state. We introduce the job data table that specifies the wafer visit sequence and the task type for each visit, as illustrated in Fig. 7, which is determined from the recipe. We let the job data table provide necessary parameters for the states or tasks in the FSM models so that more compact FSM model diagrams are obtained by reducing the number of state nodes in the diagram. In Fig. 7, process step 1 is performed by two parallel PMs, PM_1 and PM_2 . Column *Process Recipe* indicates the normal operation recipe for each process, column *Clean Recipe* specifies the recipe for auto-cleaning.

Fig. 8 shows the FSM for a PM. *WAFER READY* is an aggregated state of *SWAP* and *PUT* states. In the

Path Visit	Process Module Number			Process Recipe	Clean Recipe
Process 1	PM ₁	→	PM ₂	Proc ₁	CL _A
Process 2	PM ₃			Proc ₂	CL _A
Process 3	PM ₄				
Process 4	CL			Proc _C	CL _C

● Current PM number where the robot operation is performed

Fig. 7. The job data table for 2P4S wafer flow type.

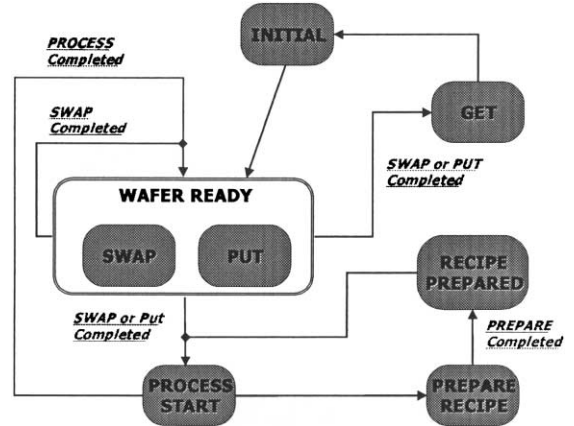


Fig. 8. The FSM model for a PM.

FSM, loading and unloading operations at a PM by the robot are represented as *PUT* and *GET* state, respectively. The FSM looks up the job data table and identifies the next task each time the state enters the aggregated state. Depending on the identified task, *SWAP* or *PUT*, the FSM determines the specific state within the aggregated state. The events are messages from the robot and the PM, which usually indicate task completions. Process completed is the process completion message from the PM, and *SWAP Completed* and *PUT Completed* are the task completion messages

from the robot. The PM controller should download the process recipe file from the tool controller and prepare the control program to execute the recipe. *PREPARE* command is issued at *PREPARE RECIPE* state to perform such preparation works prior to *PROCESS START* state.

Fig. 9 shows the FSM for the robot. Each state name represents the task name that the robot performs. *GET*, *SWAP*, and *PUT* states retrieve a wafer from the input loadlock, unloads a processed wafer from a PM and loads the next wafer in the other arm into the PM, and

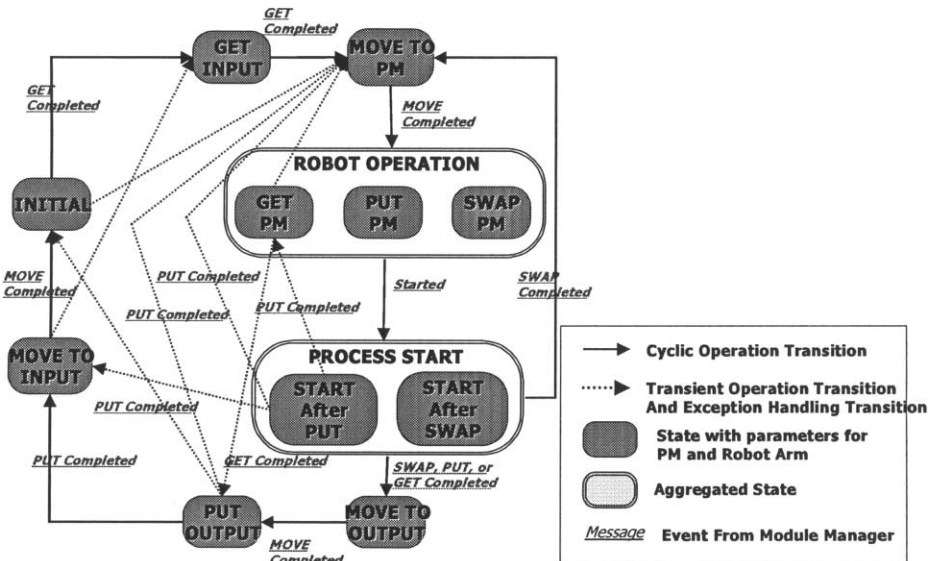


Fig. 9. The FSM model for the robot.

returns the finished wafer to the loadlock, respectively. In the normal situation, the robot repeats a cyclic operation sequence, such as *GET–SWAP–MOVE–SWAP–...–MOVE–PUT*. A *MOVE* operation that transfers the empty robot arm from a location to another one is used for advanced or auxiliary tool operations such as pipeline mode operations, single-arm operations when an arm is not available, and the initial stage operations. The robot FSM model has two aggregated states, *ROBOT OPERATION* and *PROCESS START*. The next row position at the job data table is looked up each time the state enters *ROBOT OPERATION*. Thus, the task type as one of *GET PM*, *PUT PM*, and *SWAP PM*, and the relevant PM number parameter value are determined. A similar operation method can be applied to the next aggregate state, *PROCESS START*. In each FSM model, at *INITIAL* state, the first row of the job data table is read.

Although, we do not explicitly indicate the necessary parameters in the FSM models, all states are appropriately parameterized as necessary through looking up the associated task type or the PM name in the job data table or receiving the parameters from the previous states. Such parameterization strategy using the job data table has two major benefits. First, the FSM models can be compactly represented without repeating similar states for all parameter values. For instance, we need not repeat *SWAP PM* states for all PMs. Hence, the number of states in presentation is significantly reduced and the model is better structured. Second, when the wafer flow pattern changes due to a recipe change, we need not remodel the FSMs. Our parameterized FSM model can adapt its state transitions simply by looking up the new job data table that corresponds to the new recipe.

The dotted arrows in the robot FSM represent transitions for control procedures during the initial transient period or handling procedures for exceptions such as a parallel PM failure, auto-cleaning, etc. Each time the robot FSM enters a new state, the scheduler issues the control command that corresponds to the state name. Especially, for exception handling, a code segment for conditioning and prescribing exception handling procedures can be encoded within a state. State *GET INPUT* has a procedure that computes WDI using Eq. (1) and issues a control command for unloading a wafer from the loadlock. Therefore, the scheduling logic or knowledge is inscribed into the

robot FSM. The PMs' activities follow the robot's wafer loading or unloading tasks. Therefore, in our modeling strategy, the robot can be considered as an active object or an actor that can make decision autonomously, although it is, in fact, controlled by the scheduler. An FSM that has code inscriptions on the state nodes is called an extended FSM (EFSM), popular in object-oriented modeling [18].

5. Scheduler implementation

This section explains the scheduler implementation method. The scheduler consists of an FSM executor, the FSM model, and a communication interface. The FSMs are encoded in a format that the FSM executor can recognize. The FSM executor keeps the current states of the FSMs, makes appropriate state changes each time it receives an event message from the robot or the PMs through the module manager, and sends an appropriate task command to the robot (see Fig. 10). We make appropriate communication connections between the FSM executor and the module manager. The FSM model together with the job data table incorporates the tool configuration, the wafer flow pattern, the scheduling decisions, and real-time control decisions. Therefore, it prescribes the desirable tool schedule behavior, that is, what task the robot should do for each state of the tool. The state changes and the task commands in the scheduler are made as prescribed in the FSM model. Consequently, the tool is controlled as the FSM model prescribes. Fig. 10 summarizes the concept and architecture of our FSM model-based scheduler.

Once the robot's module controller receives the control command, it performs the corresponding task. When the task is completed, the robot's module manager sends the event message for the task completion to the FSM executor. Then, the PM that is associated with the robot task performs an appropriate activity as specified in the PM's FSM model. For instance, as shown in Fig. 9, when the event message for a swap task, *SWAP Completed*, is received, the corresponding PM's FSM makes a transition to state *PROCESS START* and sends the PM a control command for starting a process. By incorporating the complex scheduling logic or desirable tool behavior in the FSM model, the program coding for scheduler

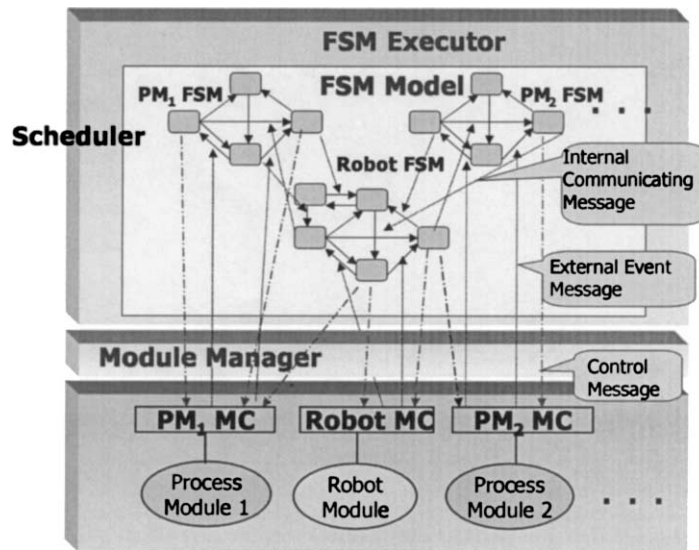


Fig. 10. The FSM model-based scheduler.

implementation is rather straightforward, better legible, and maintainable. Otherwise, the coding would be rather complicated and hard to verify.

Event messages from the module manager wait in the event queue until they are extracted and processed from the FSM executor. The event queue is implemented using distributed component object model (DCOM) of Microsoft. Commands from the FSM executor are implemented by invoking Application Program Interface (API) calls, which are defined in the module manager. When OPC (OLE for Process Control), a standard object interface for process control, including useful facilities such as variables and events, is developed to be more complete, it can be used for implementing the communication interface between the scheduler and the module manager (see [19] for OPC). Module controllers (MCs), such as PMCs and the robot MC, perform detailed subtasks when they receive a control command. For instance, when the scheduler sends the module manager a *SWAP* control command by calling the corresponding API call, the PMC and the robot MC perform a sequence of subtasks, *CHUCK UNLOAD*, *SV (Slot Valve) OPEN*, *PICK*, *ROTATE*, *PLACE*, *SV CLOSE*, and *CHUCK LOAD* (see Fig. 11). Such a subtask sequence at MCs also can be modeled as an FSM. The robot MC controls the loadlocks and the slot valves as well as the robot arms. A PMC controls the chemical process

for CVD and the associated mechanical tasks such as chuck movements within the processing chamber.

For expected exceptional events, such as a parallel module abort or auto-cleaning of a PM, appropriate handling procedures, each of which prescribes a sequence of tasks for the robot and the associated PMs, are determined by brainstorming among process engineers and equipment engineers. It is implemented by inscribing appropriate codes in the associated states of the FSMs and by adding some transition arcs as needed. Our model-based scheduler development approach facilitates such exception handling procedure implementation. When receiving an exceptional event message from the module manager, the scheduler determines whether an exception handling procedure is initiated. Event messages that indicate occurrences of exceptions include *CLEAN Required*, and *ALARM Occurred*. Fig. 12 illustrates an exception handling procedure implementation.

When an exceptional event such as an alarm occurrence is identified, the module manager notifies the occurrence to the scheduler and the FSM executor executes a predefined exception handling procedure that is inscribed in a state of the robot FSM. In Fig. 12, the current state of robot FSM is *ROBOT OPERATION* when an alarm that indicates a wafer damage occurs. The scheduler determines whether to switch *SWAP* operation to other operation like *PUT* or *GET*.

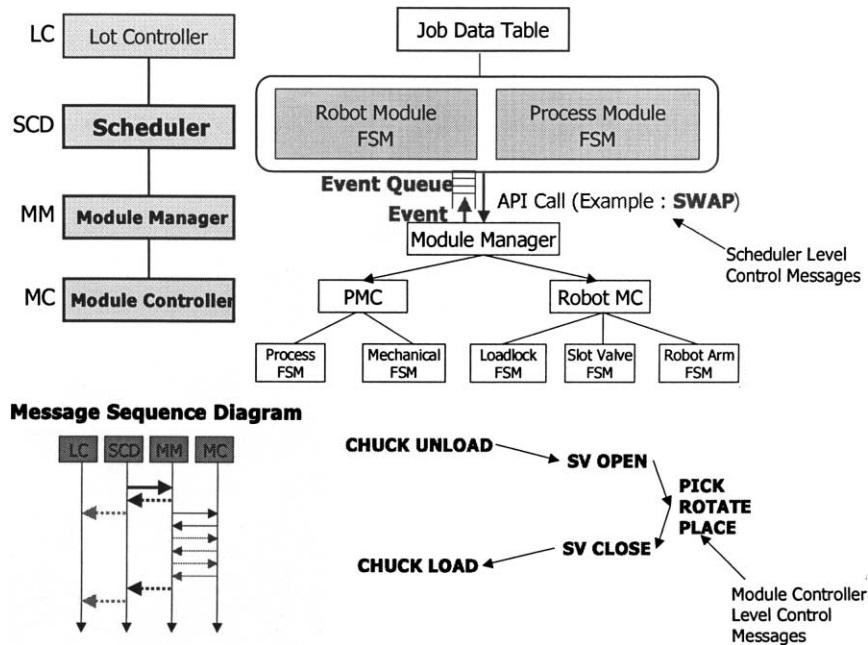


Fig. 11. Interface between the scheduler and the CTC.

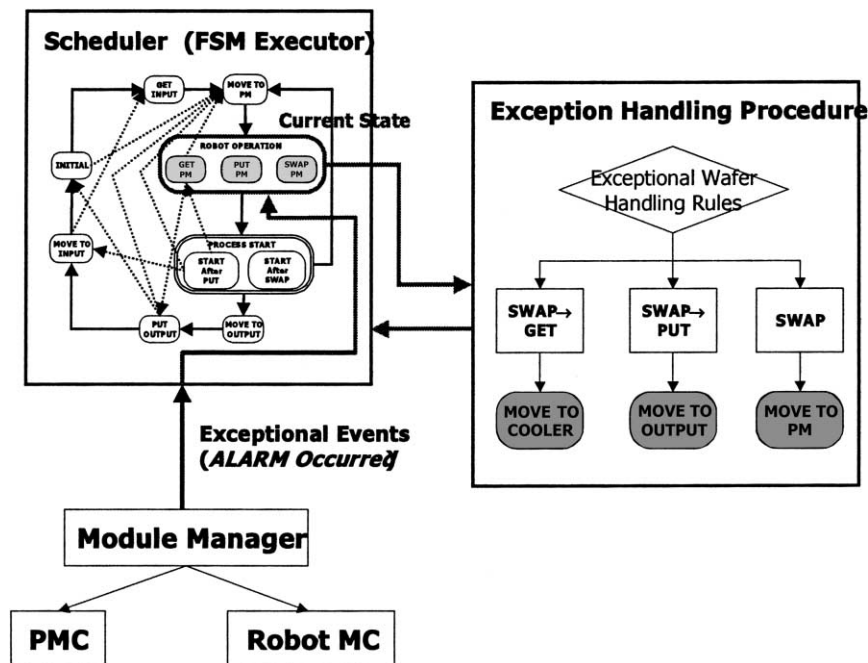


Fig. 12. An exceptional handling procedure.

The rules for handling exceptional wafers are predefined. There are three options, *RETURN TO LOADLOCK*, *STAYING AT PM*, or *GO ROUTE*. The rule, *RETURN TO LOADLOCK*, means that the wafer in the PM where an exception occurs should be returned to the output loadlock without continuing with its remaining processes. When the rule is *STAYING AT PM*, the exceptional wafer should stay at the aborted PM until an operator makes a recovery action. *GO ROUTE* rule indicates that the exceptional wafer can keep on its process route. Once an exception handling rule is applied, the robot operations and state transitions should be appropriately changed and the job data table is also updated.

6. Testing and debugging

In this section, testing and debugging methods for the scheduler will be explained. Testing and debugging the scheduler and CTC in a real cluster tool require significant time and cost. More than several hundreds of expensive wafers and process materials are wasted. Real process times are too slow to keep watching the tool behavior. Furthermore, to reduce the total tool development time by parallelizing the

development processes, we should complete the scheduler development as soon as possible before the cluster tool hardware platform is completed. To do this, we test the scheduler and CTC in a virtual tool environment. When the scheduler is integrated into the CTC, we link the CTC to virtual module controller emulators, which receive the control commands from the scheduler, mimics the behaviors of the transport module controller and the process module controller, and generates event messages at appropriate time epochs (see Fig. 13). The module controller emulators virtually generate process time or robot task time elapses and exceptional events according to a scenario. The scheduler and CTC, linked with the module controller emulators, simulate the tool operation behavior. By examining the event messages that appear in the CTC monitor screen, we verify whether the tool behavior is legitimate.

It is also possible to test and debug the scheduler model at an earlier stage before integrating it into the CTC. The FSM model may be modeled using a computer-based modeling tool such as RHAPSODY (see [20]), which can graphically model state machines, simulate the model, and transform the model into an executable code. By verifying the scheduler model in the modeling tool environment,

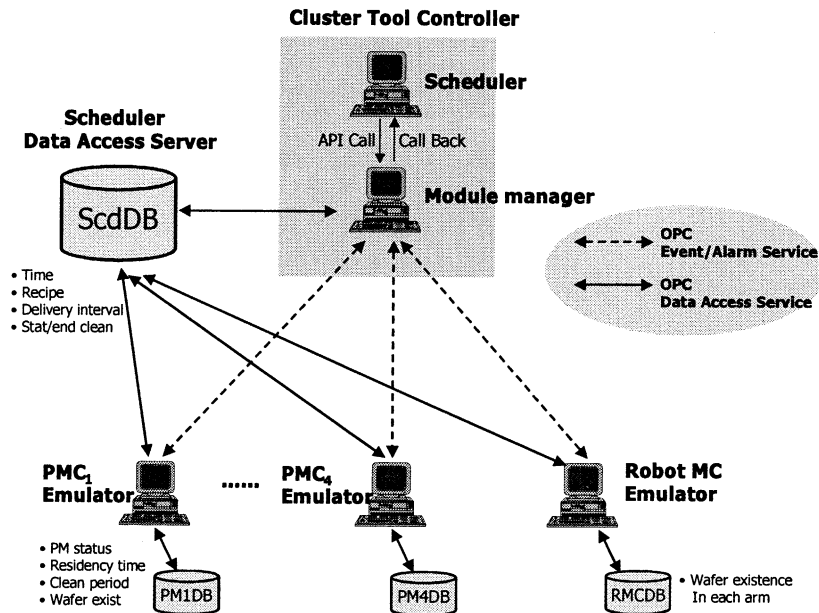


Fig. 13. Testing and debugging by a module control emulator.

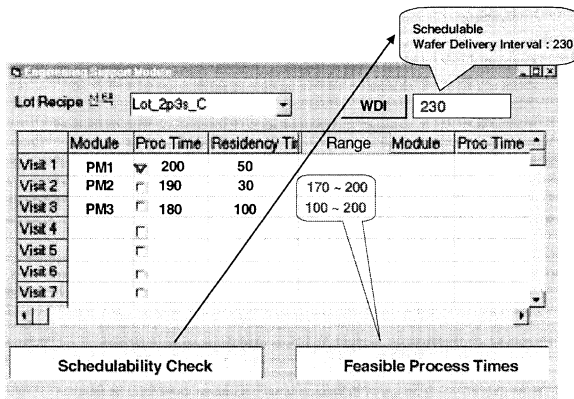


Fig. 14. A tool for schedulability check and WDI computation.

we can easily detect errors and improve the scheduler in an earlier stage. The FSM model also can be verified formally by an automata-based formal model checking tool like COSPAN, which can save time and do more complete verification than simulation methods (see [21]). This formal verification does not need an emulated plant model since it verifies the FSM model's structure and behavioral specification themselves.

Fig. 14 illustrates a software tool that checks the schedulability and computes the WDI.

The tool can check the schedulability for a given set of process times against the residency time requirement by Eq. (2). For instance, when the process times and residency time limits are input for each process step (named "visit" in the figure), by clicking the

schedulability check icon, we can identify whether there exists a feasible schedule that satisfies the residency time requirements. It also can determine the schedulable ranges of the process times for the other process steps when the process time at a process step is fixed. In the figure, when the process time for process step 1 is fixed as 200, the feasible ranges of the process times for process step 2 and 3 are computed as 170–200 and 100–200, respectively. The tool also computes the minimum schedulable WDI as the cycle time using Eq. (1), 230 in the case.

Fig. 15 summarizes the proposed development process for a real-time scheduler. The proposed procedure would be further integrated into a single unified development environment.

The advantages of our model-based approach are summarized as follows. First, a scheduling decision and a real-time control action are made at each event occurrence, and are not prescribed by a pre-computed timetable. Therefore, the controller's memory to store the timetable and rescheduling efforts are saved. The scheduler based on the FSM model can dynamically generate the control commands depending on the tool state or the occurred event. Hence, the scheduler is responsive and is desirable especially when the process time changes or exception event occurrences are frequent. Second, the real-time scheduling logic model can be flexibly modified to accommodate tool configuration or wafer flow pattern changes since the FSM model is readable and easily modified. Third, the scheduler is easily implemented by transforming the

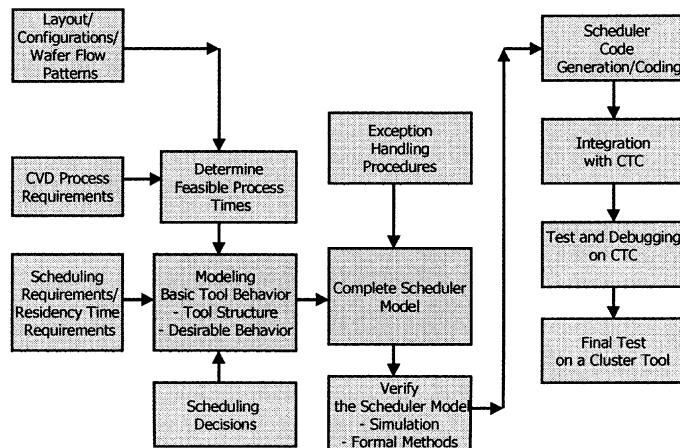


Fig. 15. The real-time scheduler development process.

FSM model into the executable code. Fourth, we can verify the FSM model before its implementation into the scheduler since the FSM model is formal and executable. Finally, FSM-based modeling appeals to modular construction or object-oriented modeling, prevalent in recent software engineering or application development.

7. Final remarks

We proposed a model-based approach for developing and implementing a real-time scheduler for dual-armed cluster tools. We discussed how the scheduling decisions for normal operation are determined based on the swap strategy. We then proposed a real-time scheduler logic model based on FSMs that not only can capture the scheduling decisions for normal operation but also can specify real-time control decisions to cope with disruptions. We also discussed how the real-time scheduler model can be implemented as a real-time scheduler in the CTC. We finally explained a debugging and testing method for the scheduler.

Our proposed method was used for developing the real-time scheduler for a real-CVD cluster tool. According to some field tests on the real tool by a cluster tool manufacturer, the scheduler showed easy adaptation against the fluctuation of process times and exceptional events. The method can evolve toward a more integrated engineering framework for developing a real-time scheduler or controller for complex manufacturing systems.

Acknowledgements

This work is partly supported by Korea Science Foundation's Grant No. 98-0200-08-01-2 and Jusung Engineering Ltd., Korea.

References

- [1] T.L. Perkinson, P.K. McLarty, R.S. Gyurcsik, R.K. Cavin Jr., Single-wafer cluster tool performance: an analysis of throughput, *IEEE Transactions on Semiconductor Manufacturing* 7 (1994) 369–373.
- [2] T.L. Perkinson, R.S. Gyurcsik, P.K. McLarty, Single-wafer cluster tool performance: an analysis of the effects of redundant chambers and revisitation sequences on throughput, *IEEE Transactions on Semiconductor Manufacturing* 9 (1996) 384–400.
- [3] H.T. LeBaron, M. Pool, The simulation of cluster tools: a new semiconductor manufacturing technology, in: *Proceedings of the 1994 Winter Simulation Conference*, 1994, pp. 907–912.
- [4] RPI, ControlPRO Developer's Guide, RealTime Performance Inc., 1996.
- [5] R. Venkatesh, S. Davenport, P. Foxhoven, J. Nulman, A steady-state throughput analysis of cluster tools: dual-blade versus single-blade robots, *IEEE Transactions on Semiconductor Manufacturing* 10 (1997) 418–424.
- [6] M.R. Comer, Scheduling method for robotic manufacturing processes, US Patent 5,801,945 (1998).
- [7] D. Jevtic, Method and apparatus for priority based scheduling of wafer processing within a multiple chamber semiconductor wafer processing tool, US Patent 5,928,389 (1999).
- [8] N.G. Hall, H. Kamoun, C. Sriskandrajah, Scheduling in robotic cells: classification, two and three machine cells, *Operations Research* 45 (1998) 421–440.
- [9] T.-E. Lee, M.E. Posner, Performance measures and schedules in periodic job shops, *Operations Research* 45 (1997) 72–91.
- [10] T. Murata, Petri Nets: properties, analysis and applications, *Proceedings of the IEEE* 77 (1989) 541–580.
- [11] H.-W. Lee, Scheduling and determination of feasible process times for CVD cluster tools with a dual end effector, M.S. Thesis, KAIST, Korea, 2000.
- [12] T.-E. Lee, Stable earliest starting schedules for cyclic job shops: a linear system approach, *International Journal of Flexible Manufacturing Systems* 12 (2000) 59–80.
- [13] A.W. Naylor, R.A. Volz, Design of integrated manufacturing system control software, *IEEE Transaction on Systems Man and Cybernetics* 17 (1987) 881–897.
- [14] S.B. Joshi, E.G. Mettala, J.S. Smith, R.A. Wysk, Formal models for control of flexible manufacturing cells: physical and system model, *IEEE Transactions on Robotics and Automation* 11 (1995) 558–570.
- [15] M.P. Fanti, B. Maione, G. Piscitelli, B. Turchiano, System approach to design generic software for real-time control of flexible manufacturing systems, *IEEE Transactions on Systems, Man and Cybernetics. Part A. Systems and Humans* 26 (1996) 190–202.
- [16] M.C. Zhou, M.D. Jeng, Modeling, analysis, simulation, scheduling and control of semiconductor manufacturing systems: A Petri net approach, *IEEE Transactions on Semiconductor Manufacturing* 11 (1998) 333–357.
- [17] SEMATECH, Executable state model-based study of SEMI's proposed mass flow controller (MFC) device specification, Technology Transfer #95032741A-TR, 1995.
- [18] J. Rumbaugh, M. Blaha, W. Permerlani, F. Eddy, W. Lorenson, *Object-Oriented Modeling and Design*, Prentice Hall, New York, 1990.
- [19] Iconics, OPC ToolWorX: Reference Guide, Iconics, 2000.
- [20] I-Logix, Rhapsody: Reference Guide, I-Logix, 2000.
- [21] T.-E. Lee, J.K. Lee, J.H. Lee, Supervisory controller design of robot assembly cells by model checking techniques, in: *Proceedings of the 20th International Conference on Computer and Industrial Engineering*, 1996, pp. 129–132.



Yong-Ho Shin is a PhD candidate in Department of Industrial Engineering, KAIST. He received his BSc degree from Seoul National University, and MSc degree from KAIST all in industrial engineering. His main research interests include semiconductor manufacturing systems, cluster tools, and automation networks.



Ja-Hee Kim is a PhD student in Department of Industrial Engineering at KAIST. She received her BSc degree and MSc degree from KAIST all in Computer Science. Her current research interests are cluster tools scheduling and project management.



Tae-Eog Lee is an Associate Professor at Department of Industrial Engineering, KAIST. He received his PhD degree in Industrial and System Engineering from The Ohio State University in 1991. His research interests include modeling, scheduling, control and integration of manufacturing systems.



Hwan-Yong Lee is a PhD student in Department of Industrial Engineering at KAIST. He received his BSc degree and MSc degree from KAIST in all industrial engineering. His current research interests are in the area of cluster tools, scheduling and modeling.